

MILITARY ASSET MANAGEMENT SYSTEM (MAMS)

TECHNICAL ARCHITECTURE & COMPREHENSIVE ASSESSMENT REPORT

Project Name: Military Asset Management System (MAMS)	Deployment: Vercel + Render + Aiven Cloud
Author/Candidate: Bhima Shankar	Repository: github.com/bhimashanka/msmrepo

1. Project Overview

The **Military Asset Management System (MAMS)** is a secure, web-based defense logistics enterprise platform designed for defense commands, base commanders, and supply officers. MAMS provides centralized, real-time tracking, allocation, procurement, inter-base transfers, and expenditure accounting for defense equipment assets including weapons, heavy vehicles, ammunition crates, and satellite communications gear.

Key System Highlights:

- **Net Movement Logistics Math:** Real-time opening balance, purchases, transfers in/out, assignments, expenditures, and closing stock calculation.
- **Role-Based Scoping (RBAC):** Scoped view enforcing strict base-level permissions for Base Commanders vs Supreme Admin oversight.
- **Full Audit Trail:** Every asset transaction generates an immutable audit record with user role, base, resource, and timestamp.
- **Dual DB Resilience Engine:** Automatic cloud database connection (Aiven PostgreSQL) with zero-config local SQLite3 and in-memory fallback capability.

System Assumptions & Limitations:

- *Assumptions:* Base IDs and Equipment categories operate under standardized defense classification codes. Multi-base transfers enforce status states (Pending, In-Transit, Completed).
- *Limitations:* Production authentication integrates JWT tokens, while demo inspector mode provides rapid single-click role switching for evaluation simplicity.

2. Tech Stack & Architecture

Layer	Technology	Architectural Rationale & Benefits
Frontend UI	React 19 + Vite 8 + TailwindCSS v4	High performance SPA, sub-second HMR development, clean military dark-mode aesthetic with interactive charts via Recharts.
Backend API	Node.js 20 LTS + Express 5	Asynchronous non-blocking REST API middleware engine, parameterized SQL queries, CORS configuration for cross-origin deployment.
Database	Aiven Cloud PostgreSQL 16	Managed relational cloud DB with SSL encryption, ACID compliance, automatic table creation DDL, and sample dataset auto-seeding.
Deployment	Vercel (Edge UI) + Render (API)	Automated Git continuous deployment: Vercel CDN for React frontend, Render web container for Node backend API.

3. Core Data Models & Schema

The database architecture consists of 9 relational tables enforcing foreign key integrity and indexing:

Entity Table	Key Attributes / Columns	Relationships & Constraints
bases	id (PK), code (UNIQUE), name, location, commander_name, created_at	Parent to inventory, purchases, transfers, assignments, expenditures, users.
equipment_types	id (PK), name, category, description, unit_of_measure, is_serialized	Classifies catalog items across Weapons, Vehicles, Ammunition, Comms.
inventory	id (PK), base_id (FK), equipment_id (FK), opening_balance, current_stock	UNIQUE(base_id, equipment_id). Tracks stock balances per base.
purchases	id (PK), base_id (FK), equipment_id (FK), quantity, unit_cost, total_cost, po_reference (UNIQUE), supplier, purchase_date	Increments inventory current_stock automatically upon creation.
transfers	id (PK), source_base_id (FK), dest_base_id (FK), equipment_id (FK), quantity, tracking_number (UNIQUE), status	Status: Pending, In-Transit, Completed. Completed transfers re-balance stock.
assignments	id (PK), base_id (FK), equipment_id (FK), quantity, personnel_name, rank, service_id, unit, status	Status: Active, Returned, Overdue. Tracks issued weapons/equipment.
expenditures	id (PK), base_id (FK), equipment_id (FK), quantity, expenditure_date, reason, operation_name	Decrements current_stock for live-fire training or combat losses.
users & audit_logs	users: id, username (UNIQUE), role, base_id (FK) audit_logs: id, username, action, resource, details, timestamp	Enforces RBAC authorization and logs immutable audit records.

4. Role-Based Access Control (RBAC)

MAMS implements a granular 3-tier Role-Based Access Control model enforced at both Express API middleware level (auth.js) and React UI component rendering level:

Role	Scope & Access Level	Enforcement Strategy
------	----------------------	----------------------

Admin (General)	Unrestricted global visibility across all bases. Full read/write access to Purchases, Transfers, Assignments, Expenditures, and Audit Logs.	authenticateUser attaches req.user.role='admin'. Global metrics aggregated across all installations.
Base Commander	Strictly scoped to their assigned Base ID (e.g. Fort Alpha HQ). Cannot initiate or view operations outside their installation.	scopeBaseAccess middleware rejects requests targeting other base IDs (HTTP 403 Forbidden). UI locks base selector.
Logistics Officer	Operational access restricted to Procurement (Purchases) and Inter-Base Transfers. Locked out of Personnel Assignments and Expenditures.	authorizeRoles('admin', 'base_commander') returns HTTP 403 for restricted endpoints.

5. Transaction Logging & Audit Trail

To ensure complete transparency and military accountability, all state-changing API transactions trigger automated audit logging through auditLogger.js middleware into the **audit_logs** database repository.

Recorded Audit Action Codes:

- **PURCHASE_RECORDED:** Logs PO reference, supplier, equipment type, quantity, total cost, and creator.
- **TRANSFER_INITIATED / COMPLETED:** Logs tracking number, origin base, destination base, equipment, and status.
- **ASSET_ASSIGNED / RETURNED:** Logs soldier personnel name, military service ID, rank, and unit.
- **ASSET_EXPENDED:** Logs expenditure operation name, ammunition/equipment quantity, and authorizing officer.

6. Setup & Installation Instructions

Local Development Setup:

```
# 1. Clone repository
git clone https://github.com/bhimashanka/msmrepo.git
cd msmrepo

# 2. Run Backend
cd backend
npm install
npm start

# 3. Run Frontend (in new terminal)
cd frontend
npm install
npm run dev
```

Production Cloud Deployment Setup:

1. **Database (Aiven PostgreSQL):** Create PostgreSQL service on Aiven, copy Service URI.
2. **Backend (Render):** Connect GitHub repo msmrepo, set Root Dir to backend, set DATABASE_URL and NODE_VERSION=20.18.0.
3. **Frontend (Vercel):** Import repo msmrepo, set Root Dir to frontend, set VITE_API_URL=https://msmrepo.onrender.com/api.

7. Key Documented API Endpoints

Method	Endpoint Route	Description & Access Level
GET	/api/health	Health check & system status timestamp.
GET	/api/bases	Fetch all military bases & outposts.
GET	/api/equipment-types	Fetch equipment catalog & categories.
GET	/api/dashboard/metrics	Calculate Net Movement, Opening/Closing stock, & Category Analytics.
GET / POST	/api/purchases	Fetch purchase logs / Record new asset acquisition.
GET / POST	/api/transfers	Fetch transfers / Initiate inter-base equipment movement.
GET / POST	/api/assignments	Fetch / Issue equipment assignments to military personnel.
GET / POST	/api/expenditures	Fetch / Record operational asset expenditures & combat losses.
GET	/api/audit-logs	Fetch system-wide transaction audit trail (Admin / Base Cmd).

8. Platform Login Credentials & Inspector Roles

The platform features instant role switching via the top navigation bar for inspector evaluation:

Role / Profile Name	Username	Assigned Base	Permitted Features
General Arthur Vance (Supreme Admin)	admin_gen	All Bases (Global)	Full Read/Write across all modules, Net Movement drill-down, Audit Trail.
Col. Marcus Miller (Base Commander)	commander_alpha	Fort Alpha HQ (Base ID 1)	Scoped operations for Fort Alpha. Cannot view or alter Bravo/Charlie/Delta data.
Col. Sarah Davis (Base Commander)	commander_bravo	Fort Bravo Post (Base ID 2)	Scoped operations for Fort Bravo Post.
Lt. James Hayes (Logistics Officer)	logistics_officer	Fort Alpha HQ (Base ID 1)	Procurement and Transfers only. Restricted from Personnel Assignments & Expenditures.